ALICE / agent_eval 2026-09-28

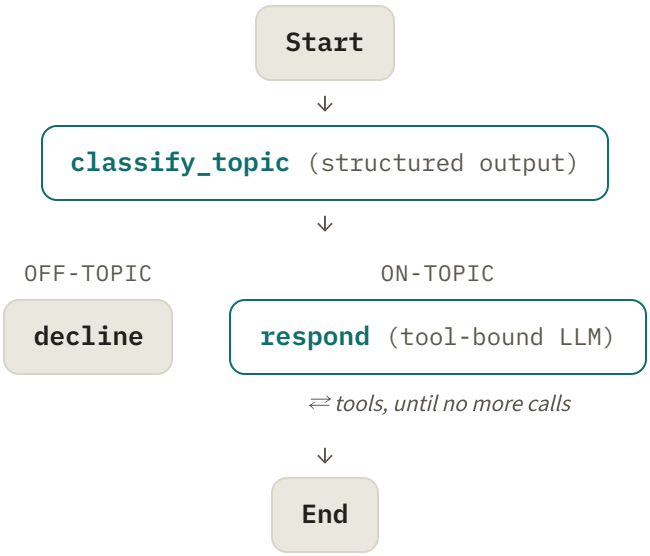
Gemma vs Qwen Benchmark

Comparing candidate local LLMs for ALICE's LangGraph trade-copilot agent, on a small eval harness. For Dean — answers to "what are you trying to do" and "how did you define the nodes and thresholds," plus a specific question on the Qwen results.

WHAT I'M DOING

ALICE is a voice copilot for Star Citizen: a LangGraph agent that classifies whether a request is on-topic, then reasons over a bound tool set (trade prices, routes, vehicle purchases, timers, etc.) to answer. It's meant to run locally on a 4080, so I need to know which model actually holds up — not just anecdotally, but against a repeatable check — before committing to one as the default.

GRAPH SHAPE



Only `classify_topic` and `respond` are swapped between runs — everything else in the graph stays fixed, so a model comparison is actually isolating the model, not the plumbing around it.

HOW THRESHOLDS ARE DEFINED

Every test case is one of three shapes, run through a fixed fake world (fake terminals, commodities, prices — no live API calls, fully reproducible) so results aren't polluted by real-world data drifting between runs:

`TOOL CASE`	Expects a specific tool + args. Checked by exact string match — deterministic, no LLM needed.
`NO-TOOL CASE`	Expects a plain reasoning answer, no tool call. Checked the same way, in reverse (asserts nothing was called).
`OFF-TOPIC CASE`	Expects `classify_topic` to correctly decline. Deterministic on/off-topic match.
`JUDGE`	A separate, fixed model (not whatever's under test) grades only the one thing code can't check: does the final response substantively convey the right information.

A case passes only if every deterministic check *and* the judge (when it runs) both pass. Pass rate is passed/total across the suite — 4 scenarios × 3 phrasings each, 12 cases per run.

RESULTS

MODEL	PASS RATE	COST / RUN	AVG LATENCY
gemma4 `(8B, current default)`	75%	$0.016	~34s
gemma4:12b	100%	$0.038	~80s
qwen2.5:14b	58%	$0.043	~91s
qwen3.5:9b	75%	$0.024	~49s

Same judge model across every run, so the comparison is apples-to-apples. Cost is priced by wall-clock GPU time × an estimated power draw × electricity rate, not tokens — there's no per-token bill running locally.

QWEN BEHAVIOR — WHERE I'D LIKE ADVICE

`qwen2.5:14b — misses the tool, then invents an answer`

On a plain "what's the cheapest place to buy X" lookup, it skipped the tool call entirely and answered from its own training data — wrong terminal, wrong price, no indication it was guessing.

> *Expected: Baijini Point, 3.15 aUEC/unit. Got: "TDD at New Babbage," 960 thousand aUEC.*

`qwen3.5:9b — perfect tool selection, confident fabrication otherwise`

9/9 correct on every tool-selection and off-topic case. But every single "explain this game mechanic" question got a confidently wrong answer, inventing mechanics that don't exist rather than answering narrowly or admitting uncertainty.

> *Invented a "Quantum Slipstream Key," claimed instant travel with no spool-up, referenced "portal gates" that aren't part of the game.*

The two failure modes look almost opposite — one under-uses tools, the other reasons fine about tools but confabulates freely on open questions. Not sure if that's a prompting fix (tighter grounding instructions), a decoding/parameter issue, or just how these particular Qwen checkpoints behave. Open to whatever angle makes sense to you.

CAVEATS

- Single run per model. Sampling variance is real here — gemma4 (8B) alone scored 10/12 then 9/12 across two identical runs, on a different case each time.
- Small dataset: 12 cases total. Enough to catch clear signal, not enough for tight confidence intervals.
- The judge is a stand-in (a local model) — the intended default judge's API key isn't usable right now, so there's a confound worth flagging even though the qwen failure *patterns* showed up consistently.

FOR DEAN

You mentioned using something for the tool-selection decision itself — cheap and quick — but the name didn't come through clearly on my end. Could you spell out what you meant?

ALICE / Graphling — agent_eval harness, gemma4 vs qwen comparison